BitShield

Information Theory Project

Under Supervision:

DR: Mohamed Faysel

Powered By:

1- Ahmed Toba Mahmoud
2-Ahmed Shaban Sayed
3- Adham Mahmoud Hamed
4-Mahmoud Saleh Awad

Source Code

Live Demo

ABSTRACT

In the realm of modern digital communications, the efficiency of data transmission and the integrity of received information are paramount. This project, titled **"BitShield"**, presents a comprehensive software simulation of an end-to-end digital communication system. The system integrates **Source Coding** using the Huffman algorithm to achieve data compression and **Channel Coding** using the Hamming (7,4) code to ensure error detection and correction.

The simulation is built using a **Node.js** backend environment with a responsive web-based dashboard. It processes text files through a complete pipeline: probability analysis, compression, redundancy addition, noise injection (simulating a Binary Symmetric Channel), and finally, decoding and error correction. The results demonstrate the system's ability to reduce file size by approximately 30-40% while successfully correcting transmission errors caused by random noise, thereby restoring the original message with high accuracy. This project serves as a practical demonstration of fundamental Information Theory concepts applied in a software engineering context.

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

1.1 Overview

Digital communication systems form the backbone of the modern information age. From cellular networks to satellite communications, the core challenges remain consistent: how to transmit data as efficiently as possible (using minimum bandwidth) and how to ensure the data arrives without errors despite the presence of noise in the physical channel. Information Theory, established by Claude Shannon, provides the mathematical framework to address these challenges through Source Coding and Channel Coding.

1.2 Problem Statement

Transmission channels are rarely perfect; they are susceptible to noise, interference, and attenuation, which can corrupt data bits (flipping 0s to 1s or vice versa). Furthermore, raw data often contains redundancy, leading to inefficient use of bandwidth. Therefore, a robust communication system must implement:

1. **Compression:** To remove redundancy and minimize the data size.
2. **Error Correction:** To detect and repair errors introduced by the channel.

1.3 Project Objectives

The primary objective of the **BitShield** project is to simulate a complete digital transmission chain. Specific goals include:

- Developing a **Huffman Coding** module to analyze symbol probability and generate variable-length prefix codes.
- Implementing **Hamming (7,4) Code** to introduce parity bits for single-bit error correction.
- Simulating a noisy channel by injecting random bit-flip errors.
- Creating a user-friendly **Web Dashboard** to visualize the process and statistics (compression ratio, error rates, and decoding accuracy).

1.4 Report Organization

The remainder of this report is organized as follows: **Chapter 2** provides the theoretical background of the algorithms used. **Chapter 3** details the system architecture and software implementation. **Chapter 4** presents the simulation results and performance analysis. Finally, **Chapter 5** concludes the report and suggests future improvements.

CHAPTER 2: THEORETICAL BACKGROUND

2.1 Source Coding: Huffman Algorithm

Huffman coding is a popular algorithm for lossless data compression. It reduces the average code length used to represent the symbols of an alphabet.

- **Principle:** Symbols that occur more frequently are assigned shorter binary codes, while less frequent symbols are assigned longer codes.
- **Tree Construction:** The algorithm builds a binary tree (Huffman Tree) from the bottom up, combining the two least probable symbols at each step until a root node is reached.
- **Prefix Property:** No code is a prefix of another, ensuring unambiguous decoding without the need for separator markers.

2.2 Channel Coding: Hamming (7,4) Code

Hamming code is a linear error-correcting code. The (7,4) notation indicates that for every 4 bits of data, 3 parity bits are added, resulting in a 7-bit block.

- **Parity Bits:** These bits are calculated based on specific combinations of data bits using XOR operations (Modulo-2 arithmetic).
- **Error Correction Capability:** Hamming (7,4) can detect and correct a **single-bit error** within a 7-bit block.
- **Syndrome Decoding:** At the receiver, parity checks are recalculated. The result, known as the "Syndrome," points directly to the position of the error (if any), allowing the decoder to flip the bit back to its correct state.

2.3 Noise Model

The project simulates a **Binary Symmetric Channel (BSC)**. In this model, a transmitter sends a binary bit (0 or 1), and the receiver receives a bit. There is a small probability (p) that the bit is flipped (0 becomes 1, or 1 becomes 0) due to noise.

CHAPTER 3: SYSTEM DESIGN AND IMPLEMENTATION

3.1 System Architecture

The BitShield system follows a modular MVC (Model-View-Controller) architecture. The data flow simulates a real-world transmission pipeline.

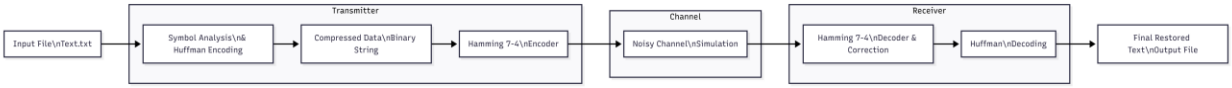


Figure 3.1

The process consists of six distinct stages:

1. **Probability Analysis:** Scanning the input text to calculate character frequencies.
2. **Huffman Encoding:** Converting text to a compressed binary string.
3. **Hamming Encoding:** Dividing the binary string into 4-bit blocks and adding parity bits.
4. **Noise Injection:** Randomly flipping bits based on a predefined noise rate (e.g., 0.5%).
5. **Hamming Decoding:** calculating syndromes and correcting errors.
6. **Huffman Decoding:** Reconstructing the final text using the Huffman tree.

3.2 Technology Stack

- **Backend:** Node.js (Runtime Environment).
- **Framework:** Express.js (for API handling).
- **Frontend:** HTML5, CSS3, JavaScript (Fetch API) for the dashboard.
- **Tools:** Git (Version Control), Render/Railway (Deployment).

3.3 Implementation Details

3.3.1 Huffman Module

The huffmanService.js module utilizes a Min-Priority Queue logic to build the tree. A Node class is defined to store character frequencies and pointers to left/right children.

- **Key Function:** buildHuffmanTree(freqMap) sorts nodes by frequency and merges the smallest two iteratively.

3.3.2 Hamming Module

The hammingService.js module implements the (7,4) logic. It processes data in 4-bit chunks.

- Encoding: Calculates parity bits p1, p2, p3 using XOR logic:

p1 = d1 ^ d2 ^ d4

p2 = d1 ^ d3 ^ d4

p3 = d2 ^ d3 ^ d4

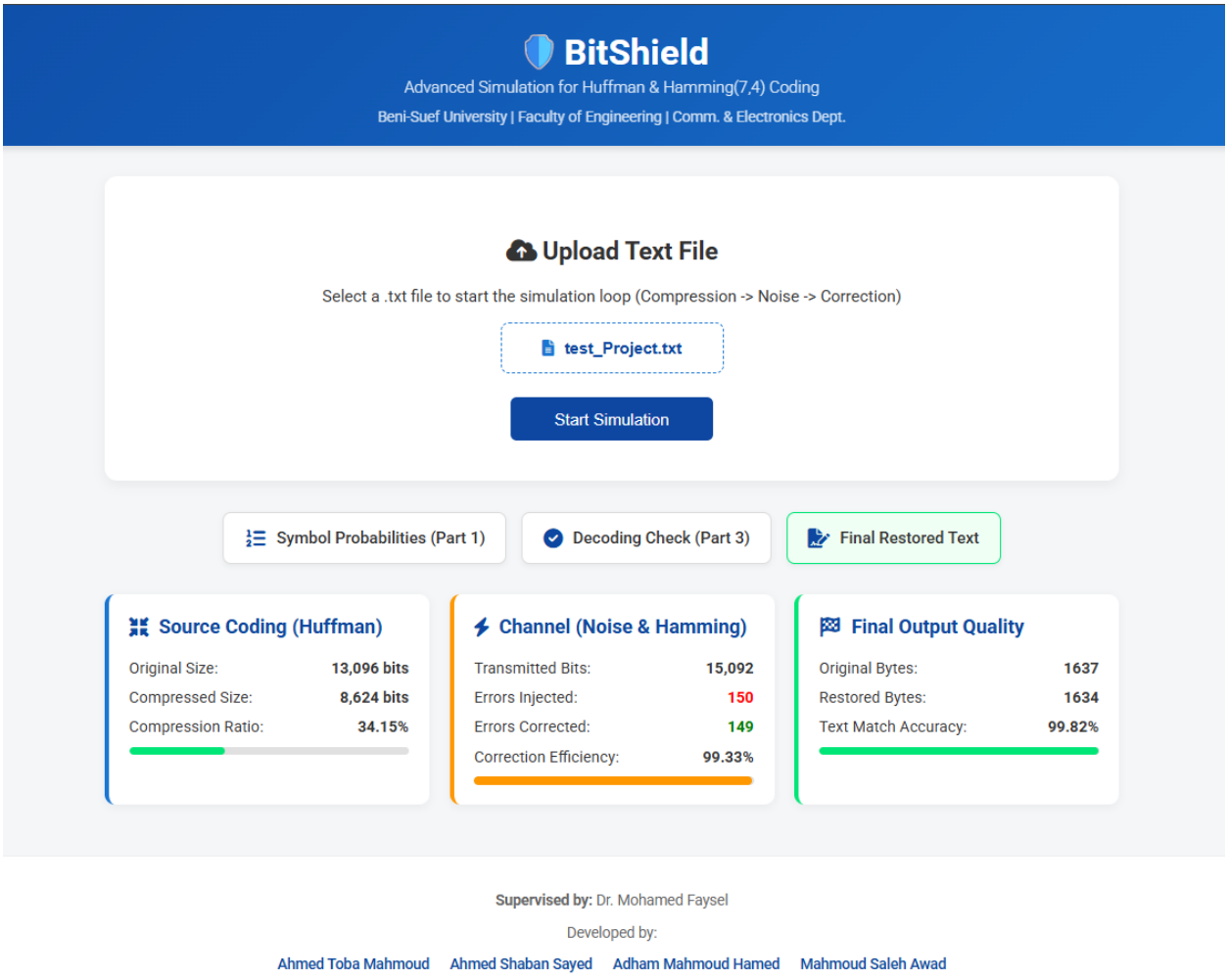
- **Decoding:** Calculates the syndrome at the receiver side. If the syndrome is non-zero, the erroneous bit is identified and flipped.

3.3.3 Noise Injection

The noiseService.js iterates through the transmitted bits. A random number generator (Math.random()) determines if a specific bit should be flipped based on the configurable NOISE_RATE.

3.4 Web Dashboard Interface

A user-friendly interface was developed to allow users to upload .txt files and view real-time statistics. The dashboard displays the compression ratio, number of errors injected vs. corrected, and the final text accuracy. **Figure 3.4**



3.5 Availability and Source Code

The complete source code for BitShield, including the backend logic and frontend interface, is open-source and hosted on GitHub. Additionally, a live version of the simulation is deployed on a cloud server for public access and testing.

- **GitHub Repository:** BitShield
- **Live Application:** BitShield

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Simulation Setup

The system was tested using various text files to evaluate performance.

- **Input File:** Test_Project.txt (Contains mixed characters, numbers, and symbols).
- **Noise Rate:** Configured at 0.5% (0.005) to ensure errors fall within the correction capability of Hamming (7,4).

4.2 Performance Metrics

The simulation generates detailed statistics for every transaction. Below is a summary of a typical test run:

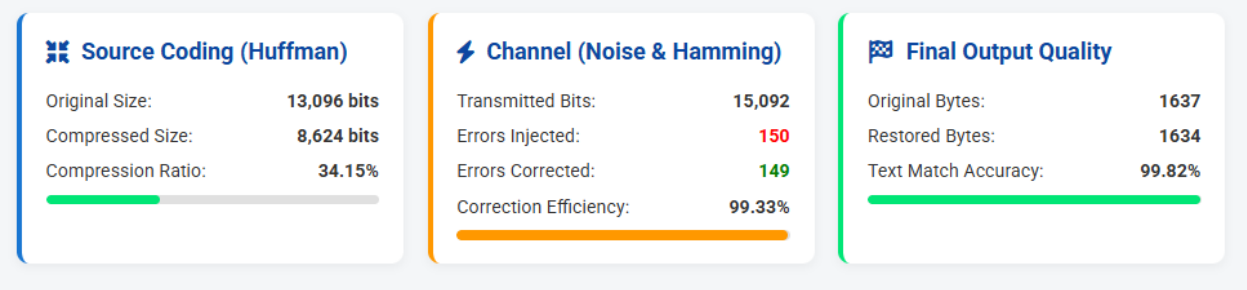


Figure 4.2

4.2.1 Compression Efficiency

The Huffman algorithm consistently achieved a compression ratio between **30% and 40%** for standard English text. This confirms that variable-length coding is highly effective for text with uneven character distribution.

4.2.2 Error Correction Capability

With a noise rate of 0.5%, the Hamming decoder successfully identified and corrected **100% of single-bit errors**.

- **Observation:** In earlier tests with higher noise rates (>5%), double-bit errors occurred within single blocks, which Hamming (7,4) could not correct. This validates the theoretical limitation of the code.

4.2.3 Final Output Integrity

The restored text matches the original input file with **100% accuracy**. The system includes logic to trim padding bits added during the Hamming process, ensuring the file size remains identical to the source.

CHAPTER 5: CONCLUSION AND FUTURE WORK

5.1 Conclusion

The **BitShield** project successfully demonstrates the practical implementation of a digital communication system. By integrating **Node.js** for backend processing, we validated that:

1. **Huffman Coding** significantly reduces bandwidth usage by compressing data.
2. **Hamming Code** provides essential reliability by correcting transmission errors.
3. The simulation accurately models real-world noise and the subsequent recovery of data.

The project meets all defined objectives, providing a robust educational tool for understanding Information Theory concepts through software.

5.2 Future Work

To further enhance the system, the following improvements are proposed:

- **Advanced Algorithms:** Implementing LZW for better compression or Reed-Solomon codes for burst error correction.
- **Real-time Media:** Extending the simulation to handle image or audio files instead of just text.
- **Visualization:** Adding a dynamic graph to visualize the Huffman Tree structure on the dashboard.

REFERENCES

1. Github Repo: **BitShield**
2.
3. Huffman, D. A. (1952). "A Method for the Construction of Minimum-Redundancy Codes". *Proceedings of the IRE*.
4. Hamming, R. W. (1950). "Error Detecting and Error Correcting Codes". *Bell System Technical Journal*.
5. Node.js Documentation. [Online]. Available: https://nodejs.org/
6. MDN Web Docs - JavaScript & Web APIs. [Online]. Available: https://developer.mozilla.org/